

Serverless для ML нагрузок

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Лавров Денис

Старший архитектор @MTS AI

- 17 лет в ИТ
- занимался архитектурой инструментов модельного инференса в Сбере
- распиливал монолиты на микросервисы
- строил аналитические хранилища и средства визуализации отчетности
- преподаватель курсов «Инфраструктурная платформа на основе Kubernetes», «Микросервисная архитектура» и «SoftwareArchitect» в ОТУС

 @roflmaoinmysoul

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#канал
группы**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Serverless

Плюсы и минусы

Serverless для ML

Воркшоп

Основные тезисы



Цели вебинара

К концу занятия вы сможете

1. Понимать что такое serverless

2. Понимать плюсы и минусы
бессерверного подхода

3. Определять для каких сценариев
подходит использование serverless

Смысл

Для чего вам это уметь

Экономить на ресурсах, сложности и
времени развертывания



**Работали ли вы уже с
serverless приложениями?**



Serverless

Serverless (FaaS) – это

облачный подход к разработке, при котором разработчик пишет только код функции, а облачный провайдер берет на себя управление серверами, масштабирование и запуск приложения. Оплата при этом производится исключительно за время работы функции.



Основные шаги

- Написать функцию на одном из поддерживаемых языков программирования
- Доставить ее в FaaS среду вашего провайдера – через UI либо автоматизированно
- Определить способ запуска – что будет являться триггером
- Получить url для вызова, если триггер это http вызов
- Получить результат работы функции, обратившись по url или инициировав триггер другим способом



Плюсы serverless

- Отсутствие необходимости тратить время и ресурсы на инфраструктуру и ее конфигурирование
- Оплата только за время работы функции
- Автоматическое масштабирование силами облачного провайдера
- Отказоустойчивость обеспечивается провайдером



Минусы serverless

- Время холодного старта (может не подходить для сервисов, требующих малого времени ответа)
- Не подходит для долгих процессов (часто ограничение не более часа)
- При активном использовании стоимость больше, чем для аренды сопоставимого железа
- Плохо подходит для statefull операций
- Vendor lock

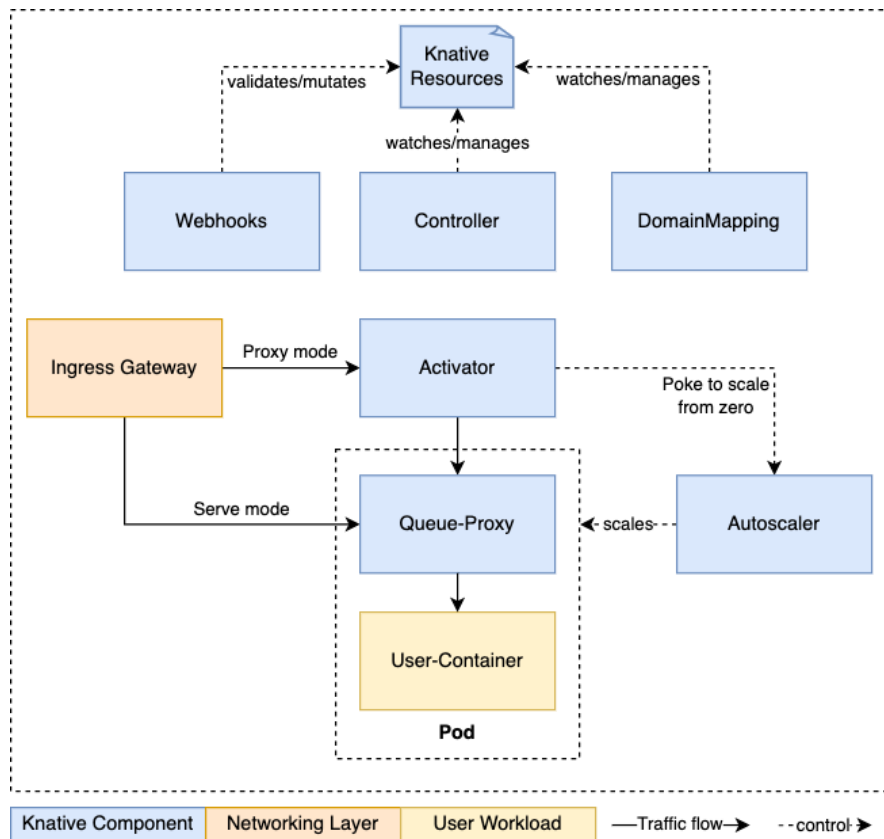


Популярные решения

- **AWS Lambda** – пионер и лидер рынка
- **Azure Functions** – второй самый крупный игрок на рынке serverless
- **Google Cloud Functions** – глубокая интеграция в инструменты Google
- **Yandex Cloud Functions** – отечественное решение, пока не имеет поддержки GPU для serverless вычислений
- **Knative** - on-prem решение, устанавливающееся в кластер K8s и позволяющее реализовать serverless подход на базе кластера
- **OpenFaaS, KEDA, Fission, KubeElasti ...**



Knative архитектура



Knative пример запуска нагрузки

Запускаем простое hello-world приложение

```
$ kn service create helloworld-go --image gcr.io/knative-samples/helloworld-go --env TARGET="World" --annotation autoscaling.knative.dev/target=10
Creating service 'helloworld-go' in namespace 'default':
0.171s Configuration "helloworld-go" is waiting for a Revision to become ready.
6.260s ...
6.324s Ingress has not yet been reconciled.
6.496s Waiting for load balancer to be ready
6.637s Ready to serve.
Service 'helloworld-go' created to latest revision 'helloworld-go-zglmv-1' is available at URL:
http://helloworld-go.default.34.71.125.175.xip.io
```

При этом подов не создается изначально

```
kubectl get pod
No resources found in default namespace.
```



Knative пример запуска нагрузки

Обращаемся к приложению и получаем ответ

```
$ curl http://helloworld-go.default.34.71.125.175.xip.io
Hello World!
```

Под созданся только после первого обращения к сервису

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
helloworld-go-zglmv-1-deployment-6d4b7fb4f-ctz86	2/2	Running	0	50s

Через некоторое время неактивности под начинает завершать свою работу

```
$ kubectl get pod -w
```

NAME	READY	STATUS	RESTARTS	AGE
helloworld-go-zglmv-1-deployment-6d4b7fb4f-d9ks6	2/2	Running	0	7s
helloworld-go-zglmv-1-deployment-6d4b7fb4f-d9ks6	2/2	Terminating	0	67s
helloworld-go-zglmv-1-deployment-6d4b7fb4f-d9ks6	1/2	Terminating	0	87s



Knative пример запуска нагрузки

Эмулируем множество запросов к приложению

```
$ hey -z 30s -c 50 http://helloworld-go.default.34.121.106.103.xip.io
Average:      0.1222 secs
Requests/sec: 408.3187
Total data:   159822 bytes
Size/request: 13 bytes
Response time histogram:
0.103 [1] |
0.444 [12243] |
0.785 [0] |
1.126 [0] |
1.467 [0] |
1.807 [0] |
2.148 [0] |
2.489 [0] |
2.830 [0] |
3.171 [0] |
3.512 [50] |
Latency distribution:
10% in 0.1042 secs
25% in 0.1048 secs
50% in 0.1057 secs
75% in 0.1077 secs
90% in 0.1121 secs
95% in 0.1192 secs
99% in 0.1826 secs
Details (average, fastest, slowest):
DNS+ Dialup: 0.0010 secs, 0.1034 secs, 3.5115 secs
DNS Lookup:  0.0006 secs, 0.0000 secs, 0.1365 secs
req write:   0.0000 secs, 0.0000 secs, 0.0062 secs
resp wait:   0.1211 secs, 0.1033 secs, 3.2698 secs
resp read:   0.0001 secs, 0.0000 secs, 0.0032 secs
Status code distribution:
[200] 12294 responses
```



Knative пример запуска нагрузки

Получаем автоматическое масштабирование количества подов под нагрузку

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
helloworld-go-thmb-1-deployment-77976785f5-6cthr	2/2	Running	0	59s
helloworld-go-thmb-1-deployment-77976785f5-7dckg	2/2	Running	0	59s
helloworld-go-thmb-1-deployment-77976785f5-fdvjn	0/2	Pending	0	57s
helloworld-go-thmb-1-deployment-77976785f5-gt55v	0/2	Pending	0	58s
helloworld-go-thmb-1-deployment-77976785f5-rwvcv	2/2	Running	0	59s
helloworld-go-thmb-1-deployment-77976785f5-tbrr7	2/2	Running	0	58s
helloworld-go-thmb-1-deployment-77976785f5-vtnz4	0/2	Pending	0	58s
helloworld-go-thmb-1-deployment-77976785f5-w8pn6	2/2	Running	0	59s



Serverless для ML

Почему может подойти для ML

- Ресурсы GPU очень дорогие для покупки и даже для постоянной 24/7 аренды
- Утилизация GPU ресурсов обычно очень низка и часто хочется платить только за время использования, а не за время простоя
- Для многих задач инференса моделей характерны не очень серьезные требования к времени ответа



Нюансы. Холодный старт

- При первом обращении сервис может отвечать достаточно долго
- ML сервисы часто загружают большие по размеру файлы моделей при старте, что еще сильнее увеличивает время старта
- Не стоит использовать для сценариев, требующих мгновенного ответа на запрос
- Для постоянного поддержания функции в памяти иногда можно применять регулярные короткие вызовы сервиса без нагрузки
- Многие провайдеры имеют опцию постоянного поддержания вашего сервиса «горячим», но это стоит дополнительных затрат и не всегда рентабельно
- Различные оптимизации для ускорения холодного старта у разных провайдеров, например хранение и запуск из снапшотов памяти запущенного сервиса



Нюансы. Время работы

- Время работы функций часто ограничено – 10-15 минут, максимум 1 час
- Таймаут API Gateway часто еще ниже, 30-60 секунд
- Время холодного старта для ML сервисов часто достаточно длительное
- Даже обработка запроса может занимать большее время, чем позволяет http таймаут
- Стоит использовать асинхронную обработку запроса, возможно с очередями и воркерами, с callback, pooling, размещением результата на S3 / в БД



Нюансы. Хранение состояния

- Функции не сохраняют состояние между запусками
- Часто стоит использовать Managed Db для хранения состояния
- Необходимо помнить о возможном исчерпании лимита соединений при резком масштабировании
- Часто имеет смысл использовать кэширующие БД вроде Redis для хранения состояния между вызовами / вызовами разных функций
- Часто используются очереди для оркестрации сложных цепочек вызова функций
- Иногда можно отдавать состояние клиенту, для использования при последующих вызовах



Нюансы. Мониторинг стоимости

- Serverless часто выгоден при низкой интенсивности обращений к сервисам, либо при «рваном» профиле нагрузки
- При стабильно высоком количестве обращений к вашему сервису serverless может выходить в несколько раз дороже, чем аренда сопоставимого оборудования
- Устанавливайте лимиты бюджета, для избежания неконтролируемых расходов
- Мониторинг потребления ресурсов
- Иногда бывает целесообразным гибридный подход, при котором нагрузка может выполняться как FaaS, так и на классической инфраструктуре



Нюансы. Безопасность

- Необходимо интегрироваться с инструментами аутентификации и авторизации, предоставляемыми вашим провайдером
- Предоставлять функциям только требующиеся им права с помощью сервис аккаунтов для каждой функции
- Иногда приемлемо дать всем функциям один общий набор прав
- Управлять безопасностью через конфигурацию VPC и сетевой изоляции



Вопросы



если есть вопросы



если вопросов нет



Практика

**Подведём
итоги занятия**

Теперь вы сможете

1. Понимать что такое serverless

2. Понимать плюсы и минусы
бессерверного подхода

3. Определять для каких сценариев
подходит использование serverless

Экономить на ресурсах, сложности и
времени развертывания



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

